

M A D 3 0 3

CI/CD best practices for building modern applications

Andrew Baird
Principal Solutions Architect
Amazon Web Services

Benjamin Susman
Lead Member of Technical Staff
Salesforce Commerce Cloud

Agenda

CI/CD foundations

CI/CD for modern applications

Continuous integration

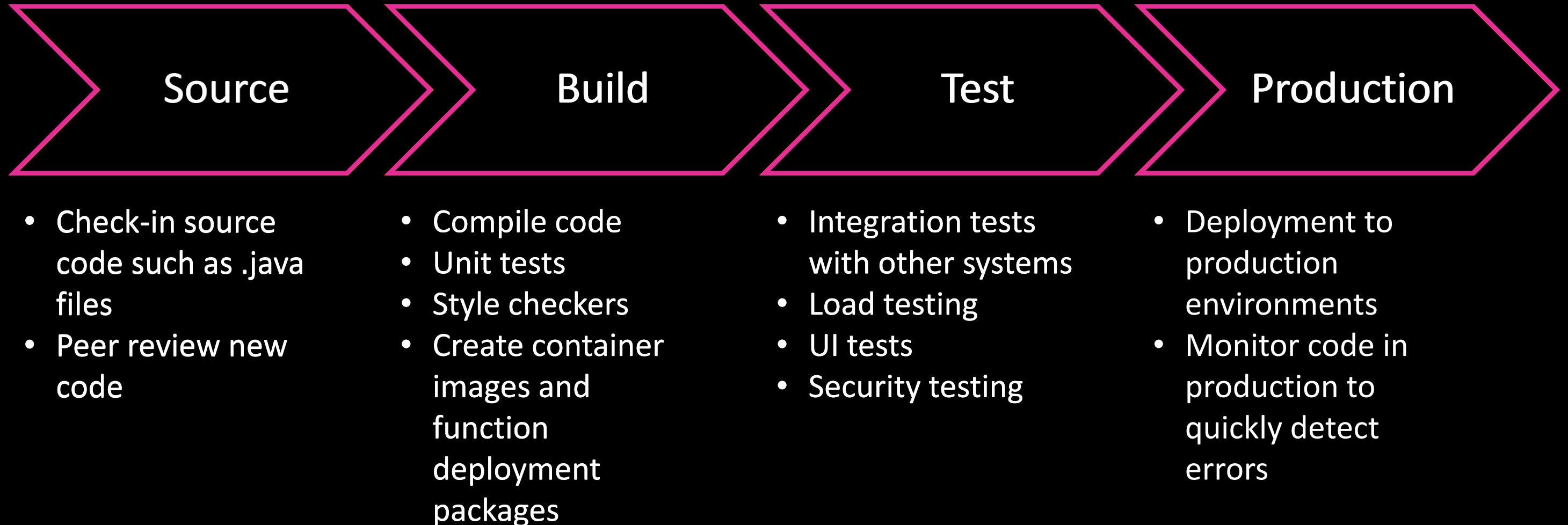
Continuous deployment

Demo

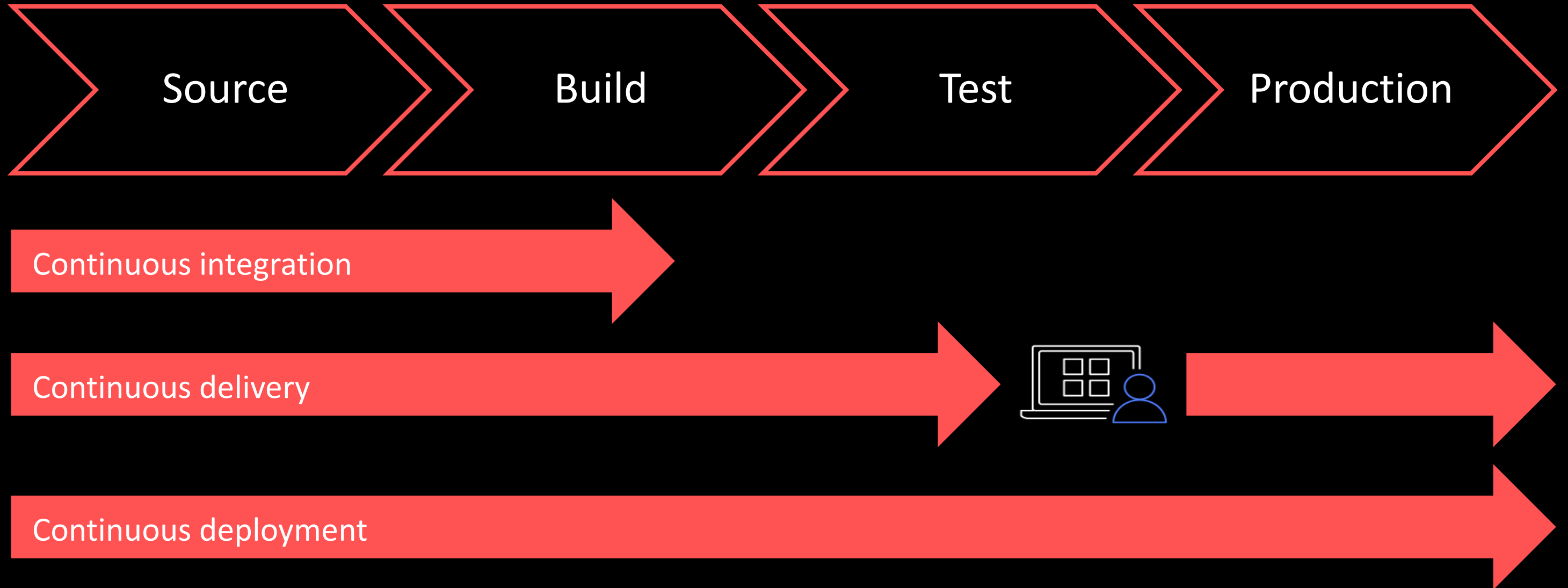
CI/CD for serverless applications – [Salesforce.com](https://salesforce.com)

CI/CD foundations

Release process stages

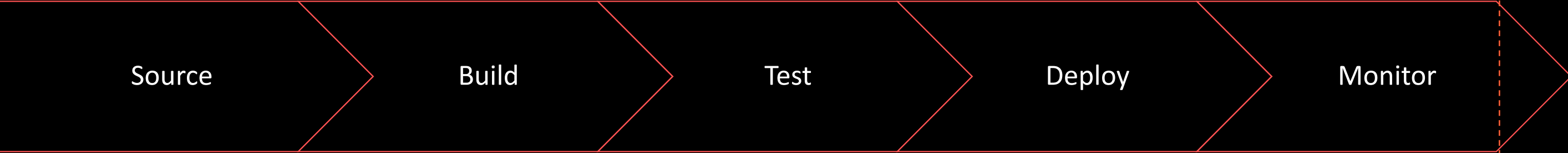
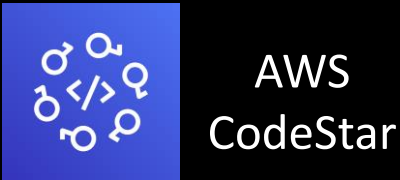


Release process stages



AWS Code services

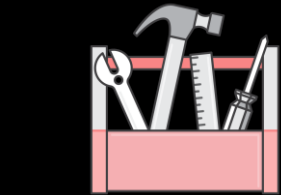
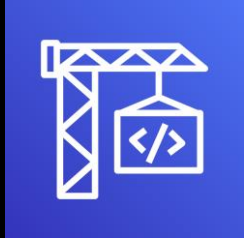
Software release steps:



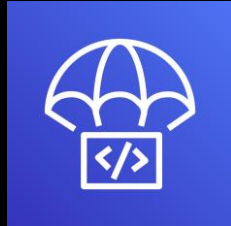
AWS CodeCommit



AWS CodeBuild



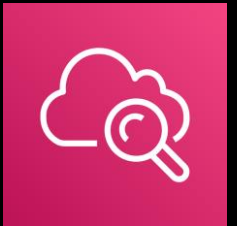
AWS CodeBuild +
third party



AWS CodeDeploy



AWS X-Ray



Amazon
CloudWatch



AWS CodePipeline

What is a modern application?

Approaches to modern application development

- Accelerate the delivery of new, high-quality services
 - Simplify environment management
 - Reduce the impact of code changes
 - Automate operations
 - Gain insight across resources and applications
 - Protect customers and the business
-

Approaches to modern application development

- Accelerate the delivery of new, high-quality services with **CI/CD**
- Simplify environment management with **serverless technologies**
- Reduce the impact of code changes with **microservice architectures**
- Automate operations by **modeling applications and infrastructure as code**
- Gain insight across resources and applications by enabling **observability**
- Protect customers and the business with **end-to-end security and compliance**

Approaches to modern application development

- Accelerate the delivery of new, high-quality services with CI/CD
- Simplify environment management with serverless technologies
- Reduce the impact of code changes with microservice architectures
- Automate operations by modeling applications and infrastructure as code
- Gain insight across resources and applications by enabling observability
- Protect customers and the business with end-to-end security and compliance

Effects of CI/CD

Source: 2018 DORA State of DevOps report

© 2019, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Effects of CI/CD

Deployment frequency

Weekly – monthly → Hourly – daily

Source: 2018 DORA State of DevOps report

© 2019, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Effects of CI/CD

Deployment frequency

Weekly – monthly



Hourly – daily

Change lead time

1–6 months



1–7 days

Source: 2018 DORA State of DevOps report

Effects of CI/CD

Deployment frequency	Weekly – monthly	→	Hourly – daily
----------------------	------------------	---	----------------

Change lead time	1–6 months	→	1–7 days
------------------	------------	---	----------

Change failure rate	46%–60%	→	0%–15%
---------------------	---------	---	--------

Source: 2018 DORA State of DevOps report

Effects of CI/CD

Deployment frequency	Weekly – monthly	→	Hourly – daily
Change lead time	1–6 months	→	1–7 days
Change failure rate	46%–60%	→	0%–15%
			
			48% of software teams

Source: 2018 DORA State of DevOps report

© 2019, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Pillars of releasing modern applications

Continuous
integration

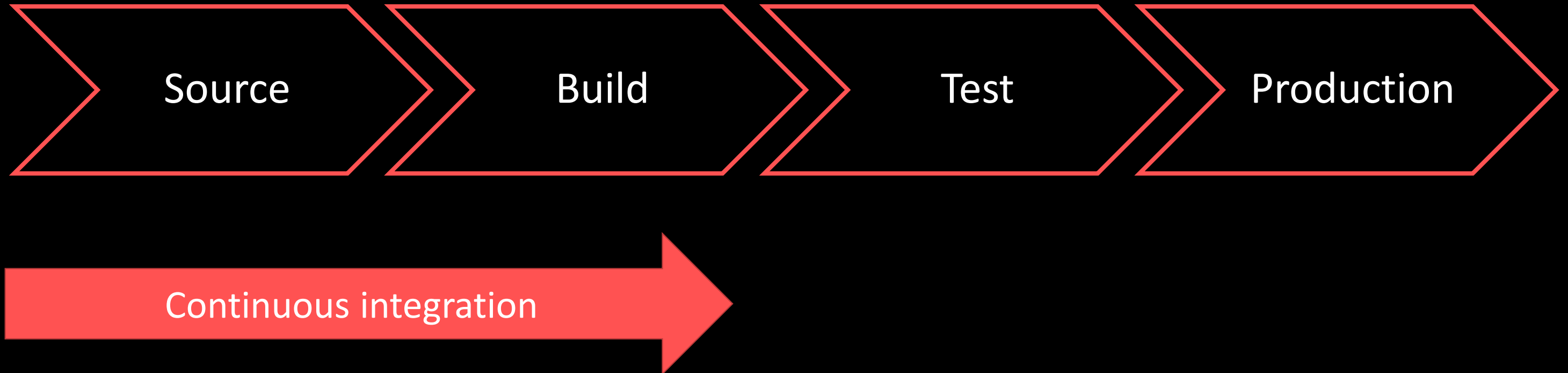
Continuous
deployment

Pillars of releasing modern applications

Continuous
integration

Continuous
deployment

Continuous integration goals



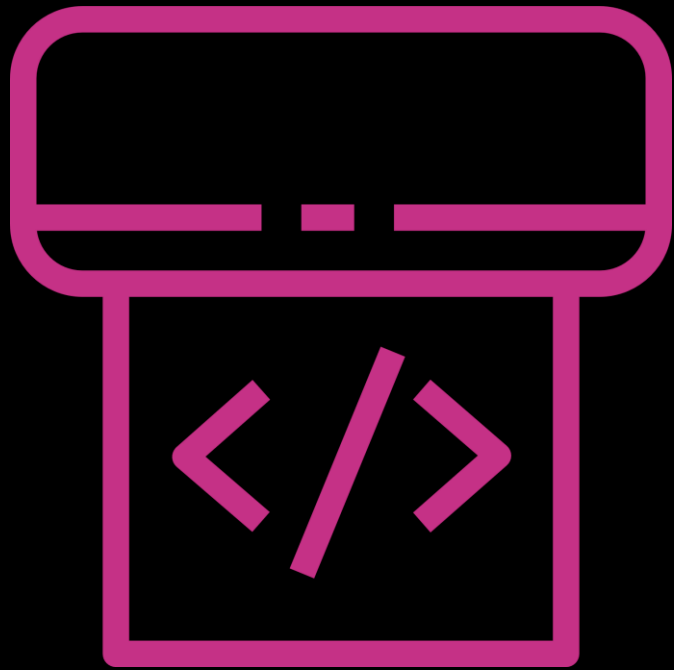
Continuous integration goals



Continuous integration

1. Automatically kick off a new build when new code is checked in
2. Build and test code in a consistent, repeatable environment
3. Continually have an artifact ready for deployment
4. Continually close feedback loop when build fails

CodePipeline



- Continuous delivery service for fast and reliable application updates
- Model and visualize your software release process
- Builds, tests, and deploys your code every time there is a code change
- Integrates with third-party tools and AWS

CodePipeline: Supported sources

Automatically kick off release and pull latest source code

Pick branch

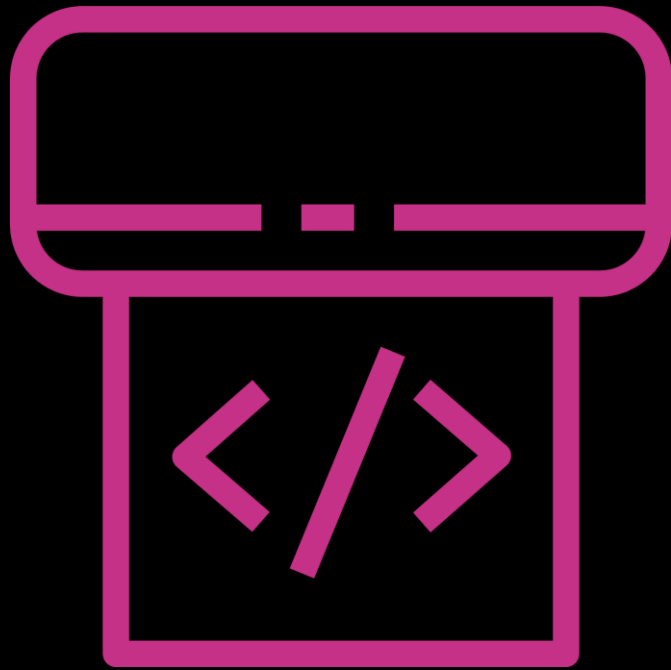
CodeCommit

GitHub

Pick object or folder

Amazon Simple
Storage Service
(Amazon S3)

CodePipeline: Supported sources



CodePipeline now uses Amazon Elastic Container Registry (Amazon ECR) as a pipeline source

CodePipeline: Supported sources

Automatically kick off release and pull latest source code

Pick branch

Pick object or folder

Pick Docker tag

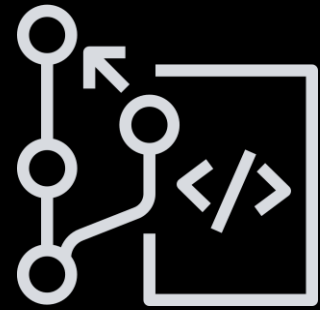
CodeCommit

Amazon S3

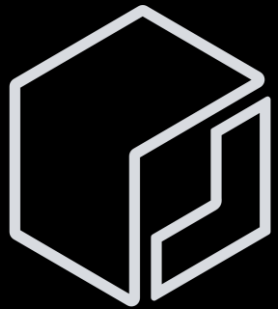
Amazon ECR

GitHub

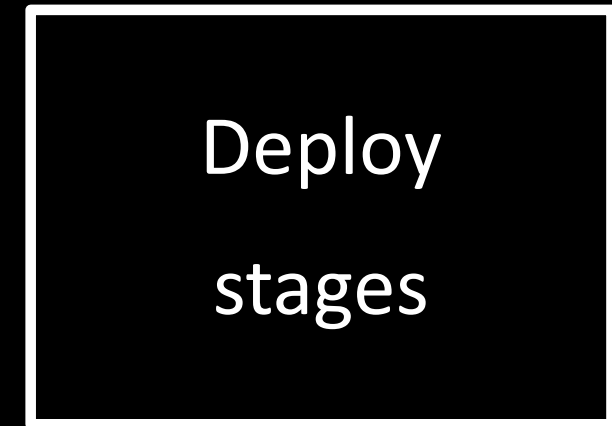
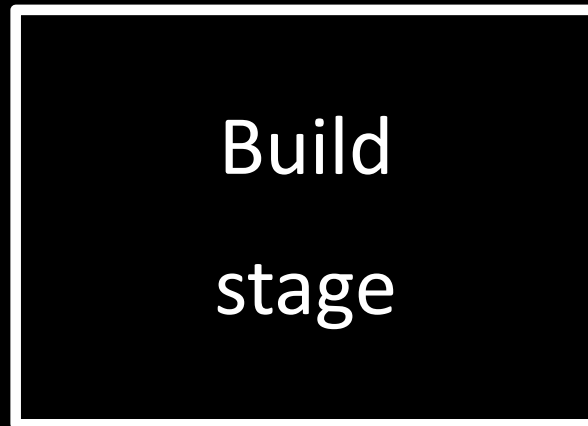
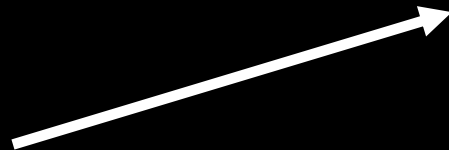
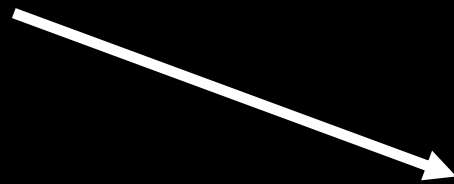
CodePipeline: Amazon ECR source action



Source code:
"master" branch



Amazon ECR
repository:
"release" tag



CodePipeline: Supported deployment targets

Automatically kick off release and pull latest source code

Amazon EC2

CodeDeploy

AWS Elastic Beanstalk

AWS OpsWorks stacks

Containers

CodeDeploy

Amazon ECS

Amazon ECS (blue/green)

AWS Fargate

Serverless

CodeDeploy

AWS CloudFormation
(AWS SAM)

AWS Lambda

CodePipeline: Supported triggers

Automatically kick off release

Amazon CloudWatch Events

- Scheduled (nightly release)
- AWS Health events (Fargate platform retirement)

Available in CloudWatch Events console, API, SDK, CLI, and AWS CloudFormation

Webhooks

- Docker Hub
- Quay
- Artifactory

Available in CodePipeline API, SDK, CLI, and AWS CloudFormation

Approaches to modern application development

- Accelerate the delivery of new, high-quality services with CI/CD
 - Simplify environment management with **serverless technologies**
 - Reduce the impact of code changes with **microservice architectures**
 - Automate operations by modeling applications and infrastructure as code
 - Gain insight across resources and applications by enabling observability
 - Protect customers and the business with end-to-end security and compliance
-

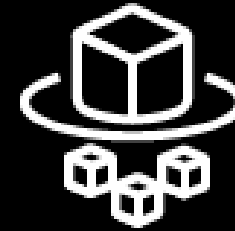
Approaches to modern application development



AWS Lambda

Serverless functions

- Event-driven
- Many language runtimes
- Data source integrations
- No server management

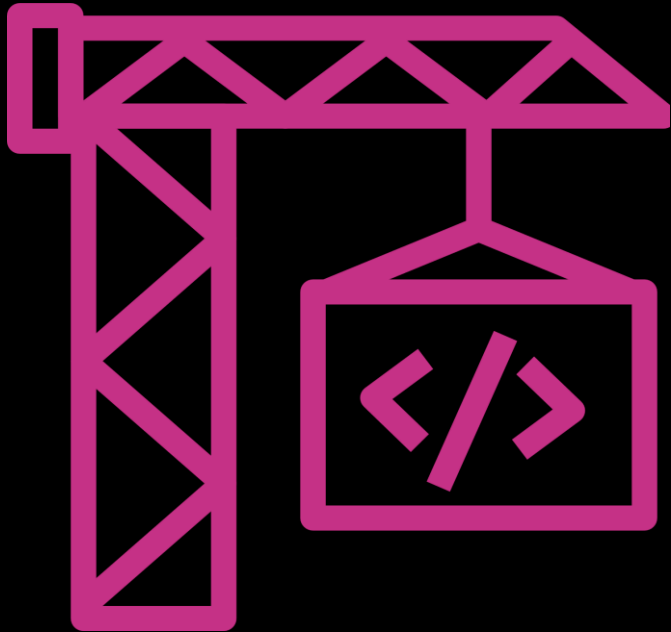


AWS Fargate

Serverless containers

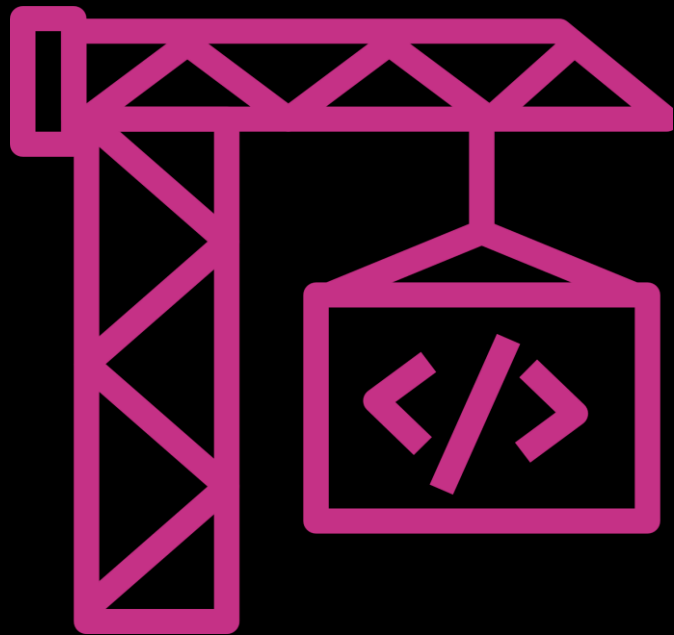
- Long-running
- Abstracts the OS
- Fully managed orchestration
- Fully managed cluster scaling

CodeBuild



- Fully managed build service that compiles source code, runs tests, and produces software packages
- Scales continuously and processes multiple builds concurrently
- No build servers to manage
- Pay by the minute, only for the compute resources you use
- Monitor builds through CloudWatch Events

CodeBuild



- Each build runs in a new Docker container for a consistent, immutable environment
- Docker and AWS CLI are installed in every official CodeBuild image
- Provide custom build environments suited to your needs through the use of Docker images

CodeBuild: Lambda build specification

version: 0.2

phases:

build:

commands:

- npm ci
- npm test
- >

aws cloudformation package

--template-file template.yml

--output-template template-output.yml

--s3_bucket \$BUCKET

artifacts:

type: zip

files:

- template-output.yml

CodeBuild: Docker build specification

version: 0.2

phases:

 build:

 commands:

- \$(aws ecr get-login --no-include-email)
- docker build -t \$IMAGE_REPO_NAME:\$IMAGE_TAG .
- docker tag \$IMAGE_REPO_NAME:\$IMAGE_TAG \$ECR_REPO:\$IMAGE_TAG
- docker push \$ECR_REPO:\$IMAGE_TAG

Continuous-integration goals



Continuous integration

1. Automatically kick off a new build when new code is checked in
2. Build and test code in a consistent, repeatable environment
3. Continually have an artifact ready for deployment
4. Continually close feedback loop when build fails

Pillars of releasing modern applications

Continuous
integration

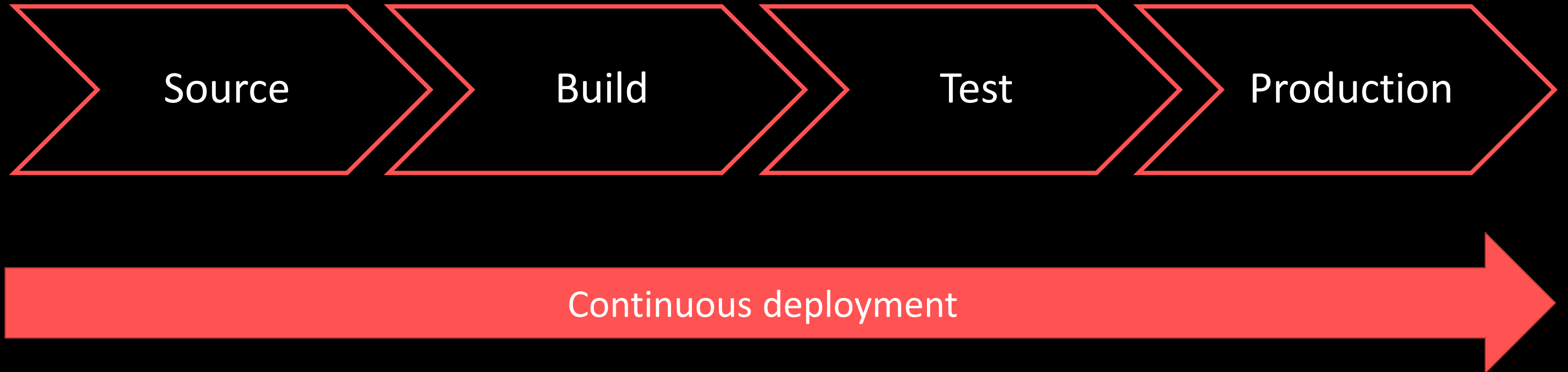
Continuous
deployment

Pillars of releasing modern applications

Continuous
integration

Continuous
deployment

Continuous-deployment goals



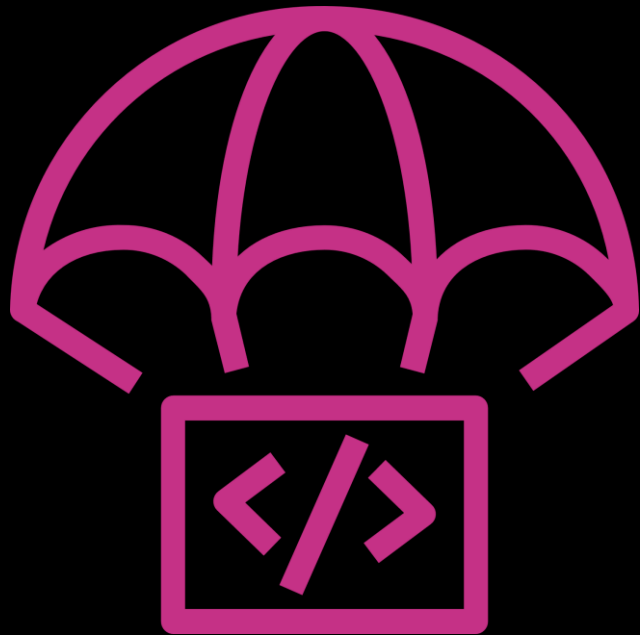
Continuous-deployment goals



Continuous deployment

1. Automatically deploy new changes to staging environments for testing
2. Deploy to production safely without impacting customers
3. Deliver to customers faster: Increase deployment frequency, and reduce change lead time and change failure rate

CodeDeploy



- Automates code deployments to any instance and Lambda
- Handles the complexity of updating your applications
- Avoid downtime during application deployment
- Roll back automatically if failure detected
- Deploy to Amazon EC2, Amazon ECS, Lambda, or on-premises servers

CodeDeploy-Amazon EC2 deployments

version: 0.0

os: linux

files:

- source: /
destination: /var/www/html

permissions:

- object: /var/www/html
pattern: "*.html"
owner: root
group: root
mode: 755

hooks:

ApplicationStop:

- location: scripts/deregister_from_elb.sh

BeforeInstall:

- location: scripts/install_dependencies.sh

ApplicationStart:

- location: scripts/start_httpd.sh

validateService:

- location: scripts/test_site.sh
- location: scripts/register_with_elb.sh

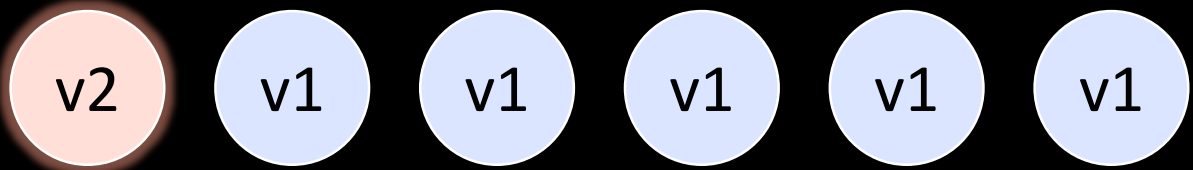
CodeDeploy-Amazon EC2 deployments

```
version: 0.0
os: linux
files:
  - source: /
    destination: /var/www/html
permissions:
  - object: /var/www/html
    pattern: "*.html"
    owner: root
    group: root
    mode: 755
hooks:
  ApplicationStop:
    - location: scripts/deregister_from_elb.sh
  BeforeInstall:
    - location: scripts/install_dependencies.sh
  ApplicationStart:
    - location: scripts/start_httpd.sh
  validateService:
    - location: scripts/test_site.sh
    - location: scripts/register_with_elb.sh
```

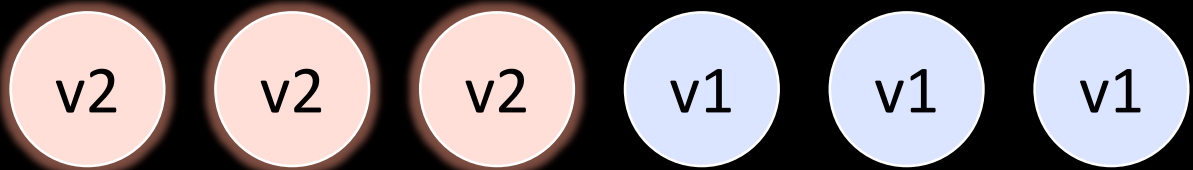
- Send application files to one directory and configuration files to another
- Set specific permissions on specific directories and files
- Remove/add instance to Elastic Load Balancing
- Install dependency packages
- Start Apache
- Confirm successful deploy
- More!

Choose deployment speed and group

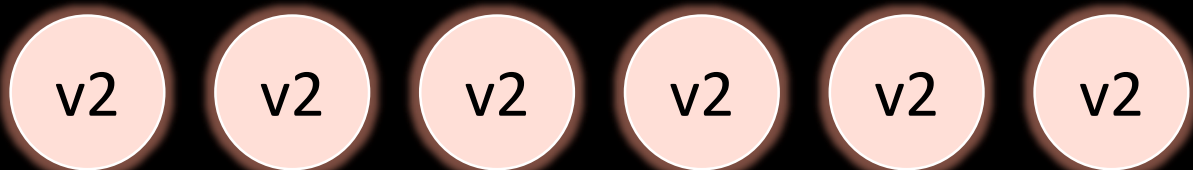
One at a time



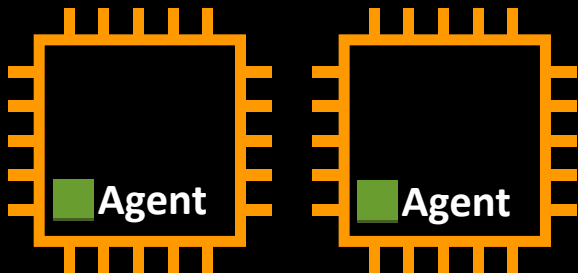
Half at a time



All at once

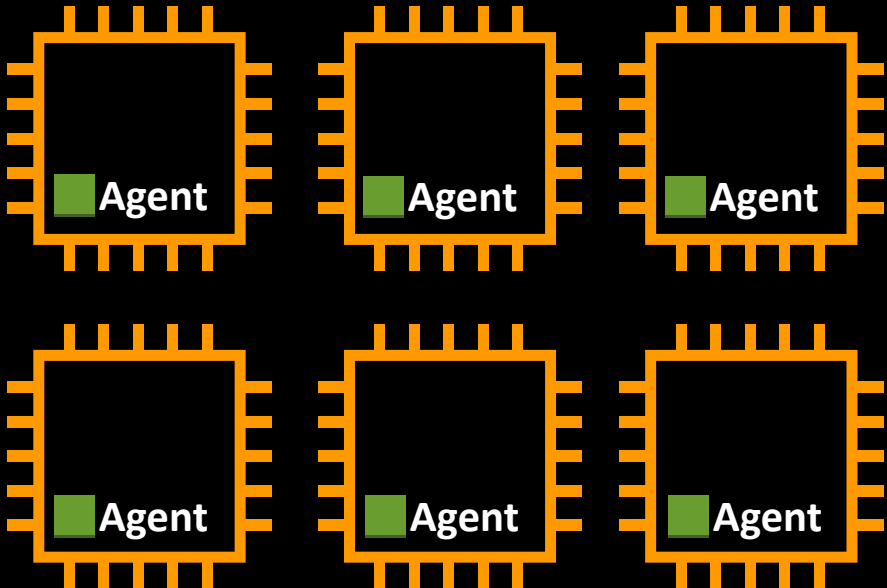


Dev deployment group



OR

Prod deployment group



CodeDeploy-Lambda deployments

- Shifts traffic using Lambda function weighted aliases
- Choose canary (“shift 10% of traffic for 10 minutes, then shift rest”) or linear (“shift 10% more traffic every 10 minutes”)
- Validation “hooks” enable testing at each stage of the deployment
- Fast rollback in seconds if case of hook failure or CloudWatch alarms
- Monitor deployment status and history via console, API, Amazon Simple Notification Service (Amazon SNS) notifications, and CloudWatch Events

CodeDeploy-Lambda deployments

Enable in your serverless application template

Resources:

 GetFunction:

 Type: AWS::Serverless::Function

 Properties:

 DeploymentPreference:

 Type: Canary10Percent10Minutes

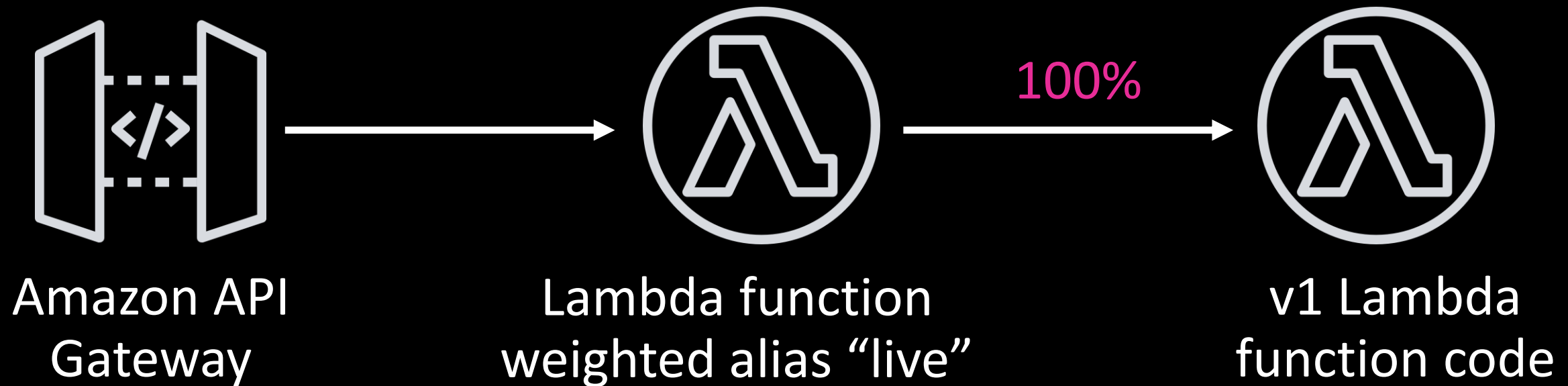
 Alarms:

 - !Ref ErrorsAlarm

 Hooks:

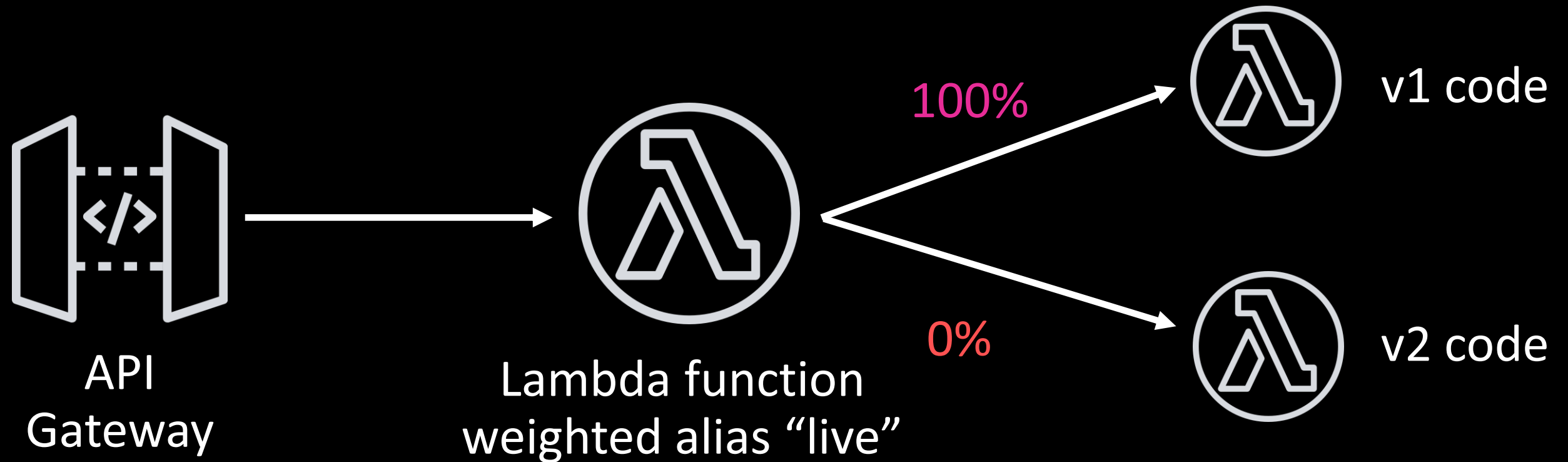
 PreTraffic: !Ref PreTrafficHook

CodeDeploy-Lambda canary deployment



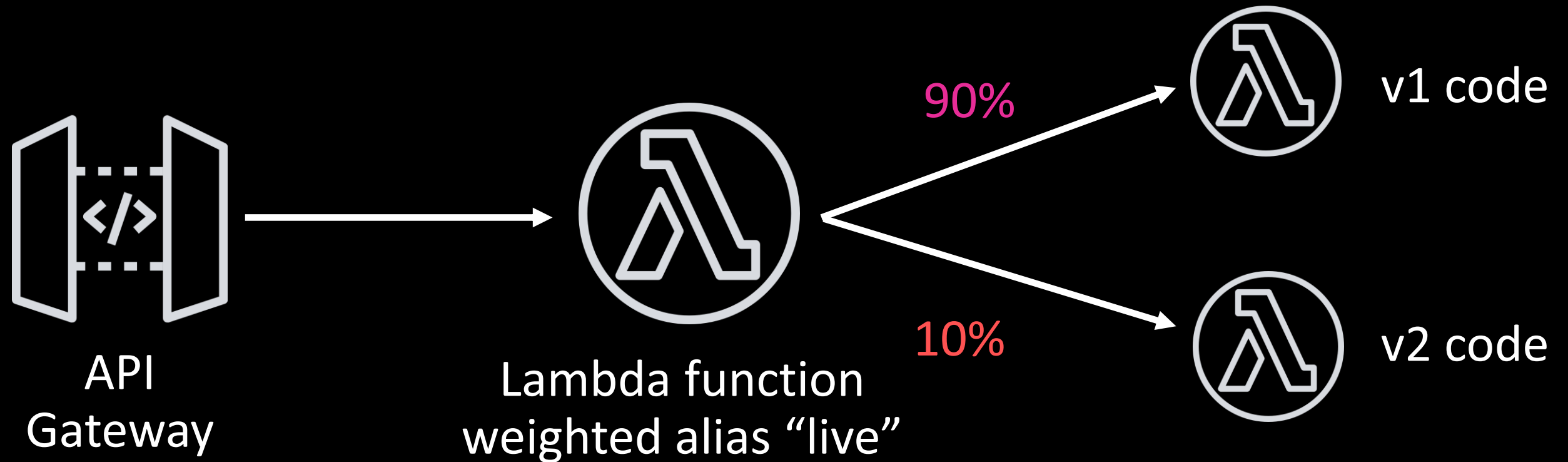
CodeDeploy-Lambda canary deployment

Run hook against v2 code before it receives traffic



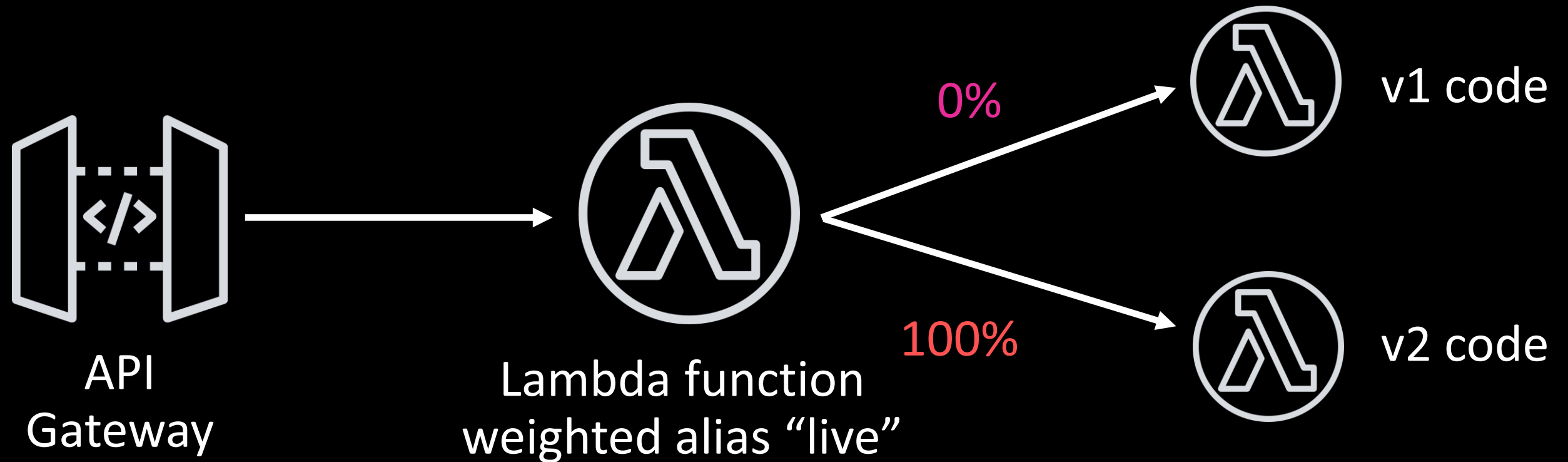
CodeDeploy-Lambda canary deployment

Wait for 10 minutes, roll back in case of alarm

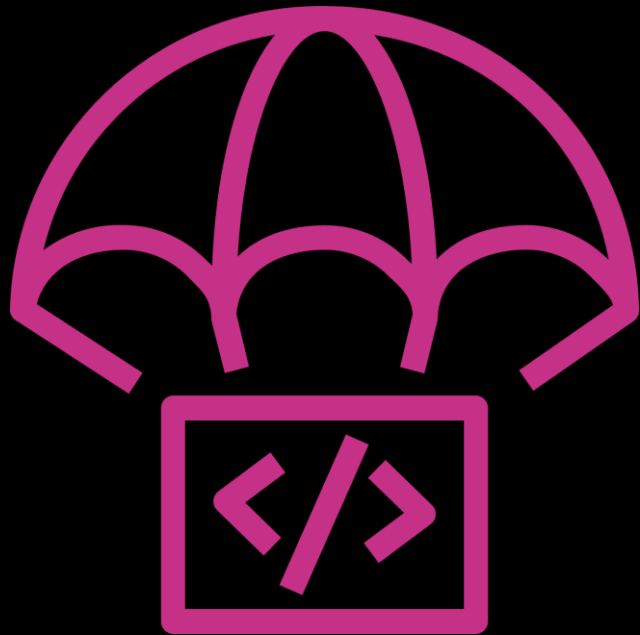


CodeDeploy-Lambda canary deployment

Complete deployment



CodeDeploy



CodeDeploy now automates
blue/green deployments to Fargate
and Amazon Elastic Container Service
(Amazon ECS)

CodeDeploy-Amazon ECS blue/green deployments

- Provisions “green” tasks, then flips traffic at the load balancer
- Validation “hooks” enable testing at each stage of the deployment
- Fast rollback to “blue” tasks in seconds in case of hook failure or CloudWatch alarms
- Monitor deployment status and history via console, API, Amazon SNS notifications, and CloudWatch Events
- Use “CodeDeploy-ECS” deploy action in CodePipeline or “aws ecs deploy” command in Jenkins

CodeDeploy-Amazon ECS AppSpec

version: 1.0

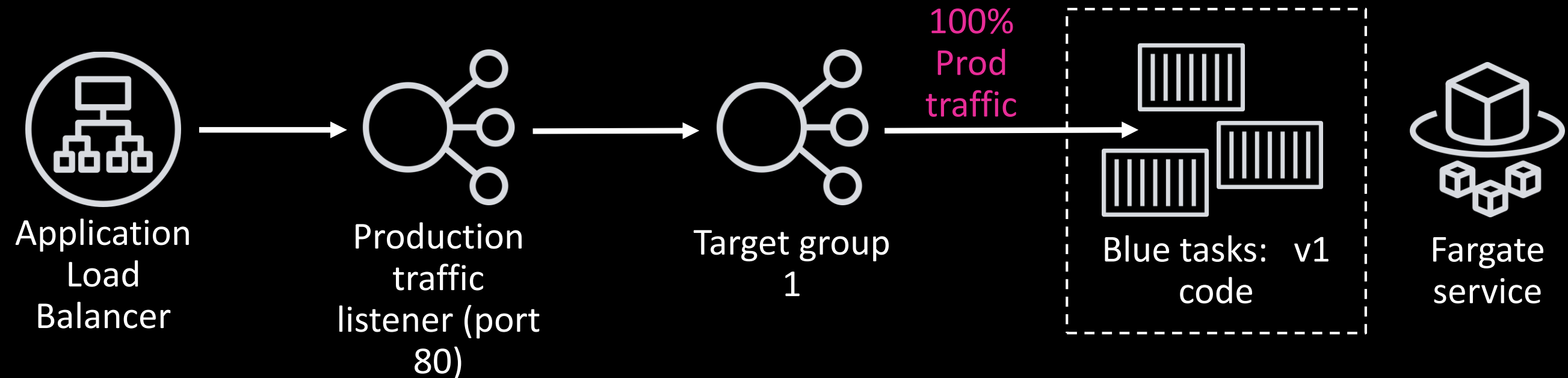
Resources:

- TargetService:
 - Type: AWS::ECS::Service
 - Properties:
 - TaskDefinition: "my_task_definition:8"
 - LoadBalancerInfos:
 - ContainerName: "SampleApp"
 - ContainerPort: 80

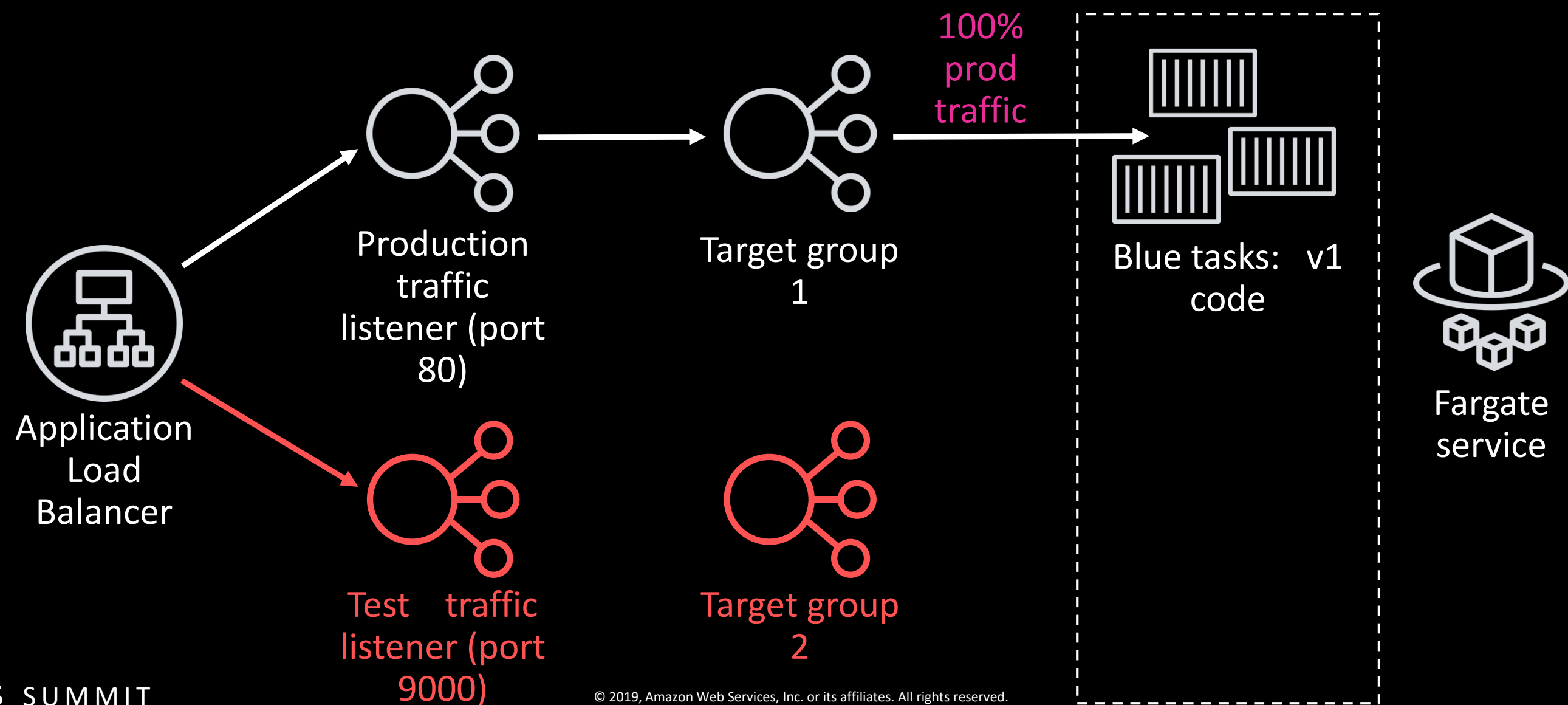
Hooks:

- BeforeInstall: "LambdaFunctionToExecuteAnythingBeforeNewRevisionInstallation"
- AfterInstall: "LambdaFunctionToExecuteAnythingAfterNewRevisionInstallation"
- AfterAllowTestTraffic: "LambdaFunctionToValidateAfterTestTrafficShift"
- BeforeAllowTraffic: "LambdaFunctionToValidateBeforeTrafficShift"
- AfterAllowTraffic: "LambdaFunctionToValidateAfterTrafficShift"

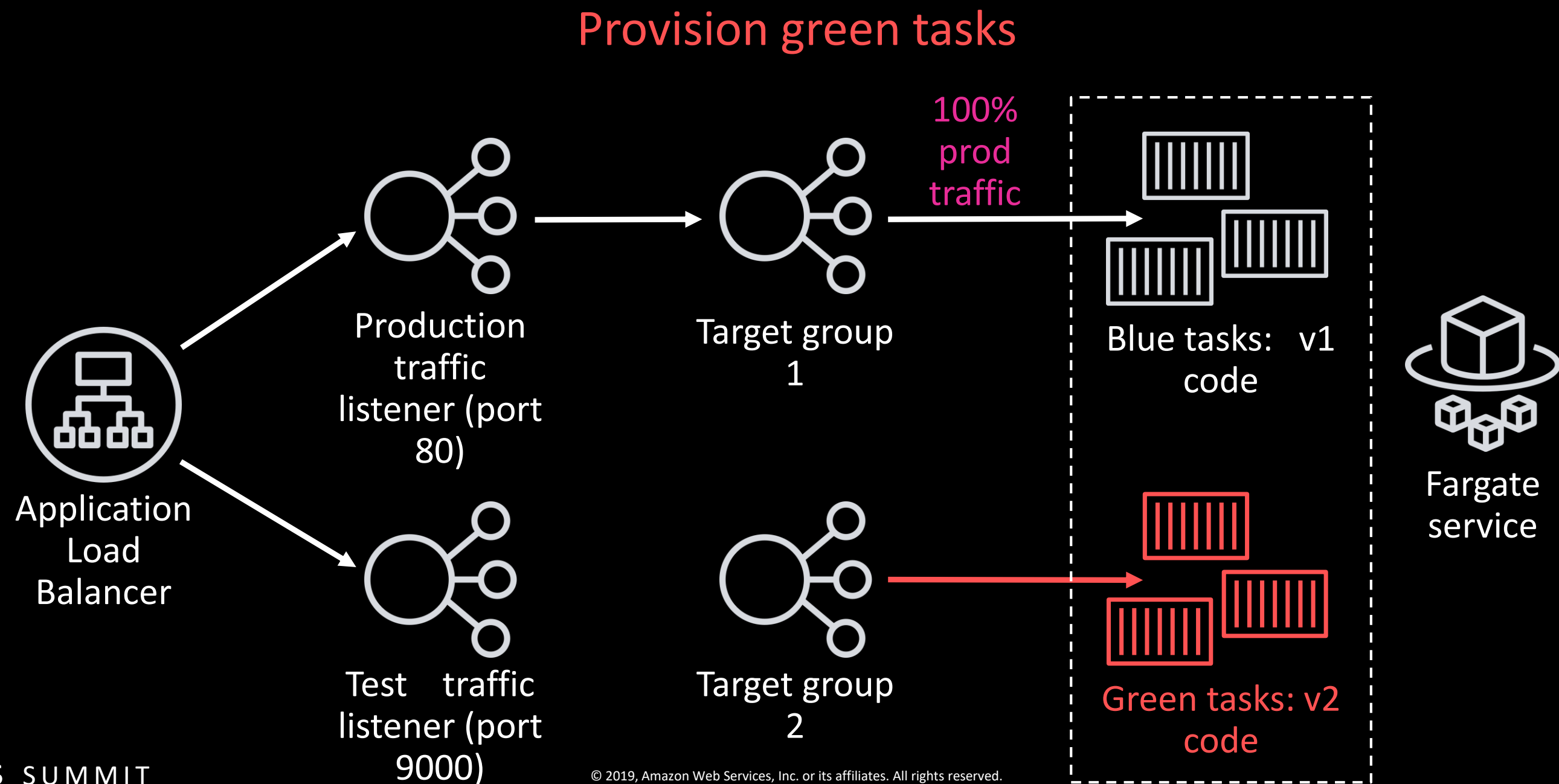
CodeDeploy-Amazon ECS blue/green deployment



CodeDeploy-Amazon ECS blue/green deployment

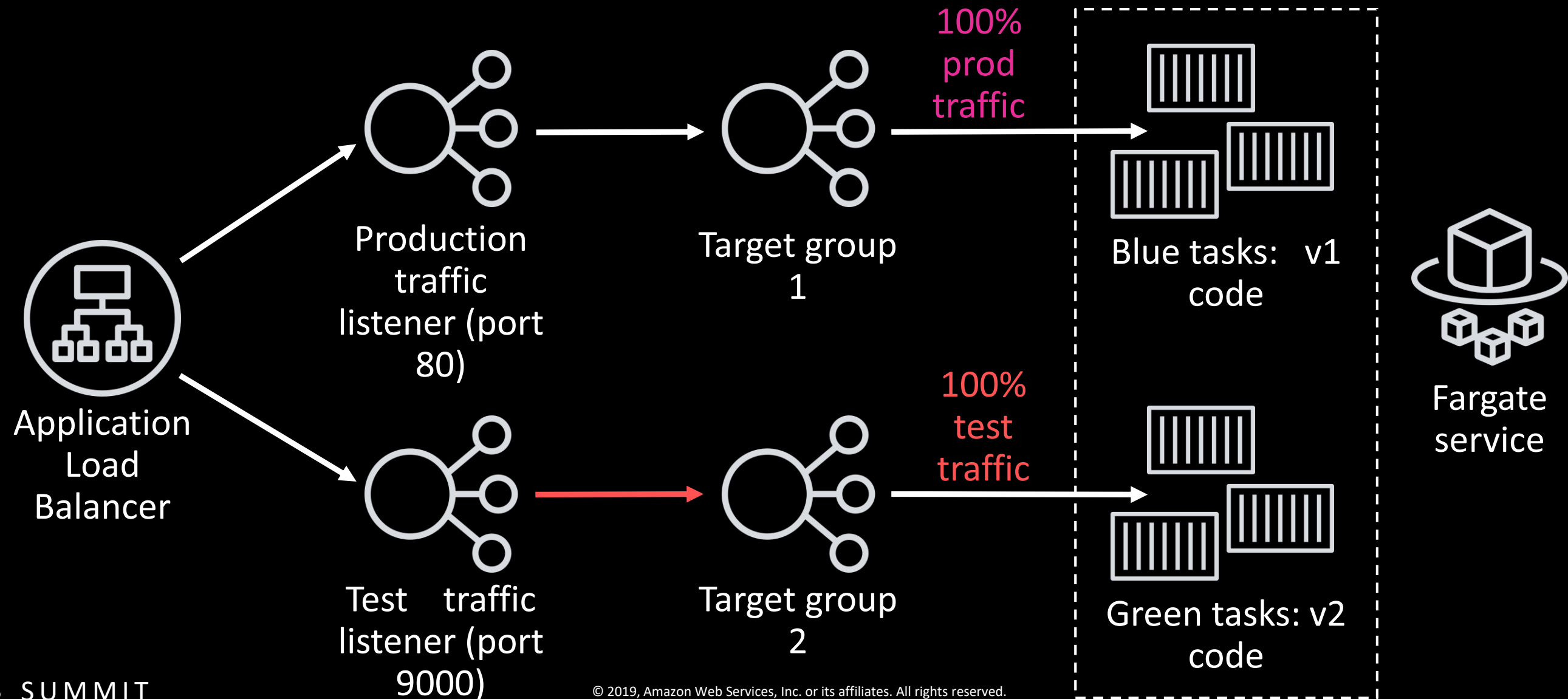


CodeDeploy-Amazon ECS blue/green deployment



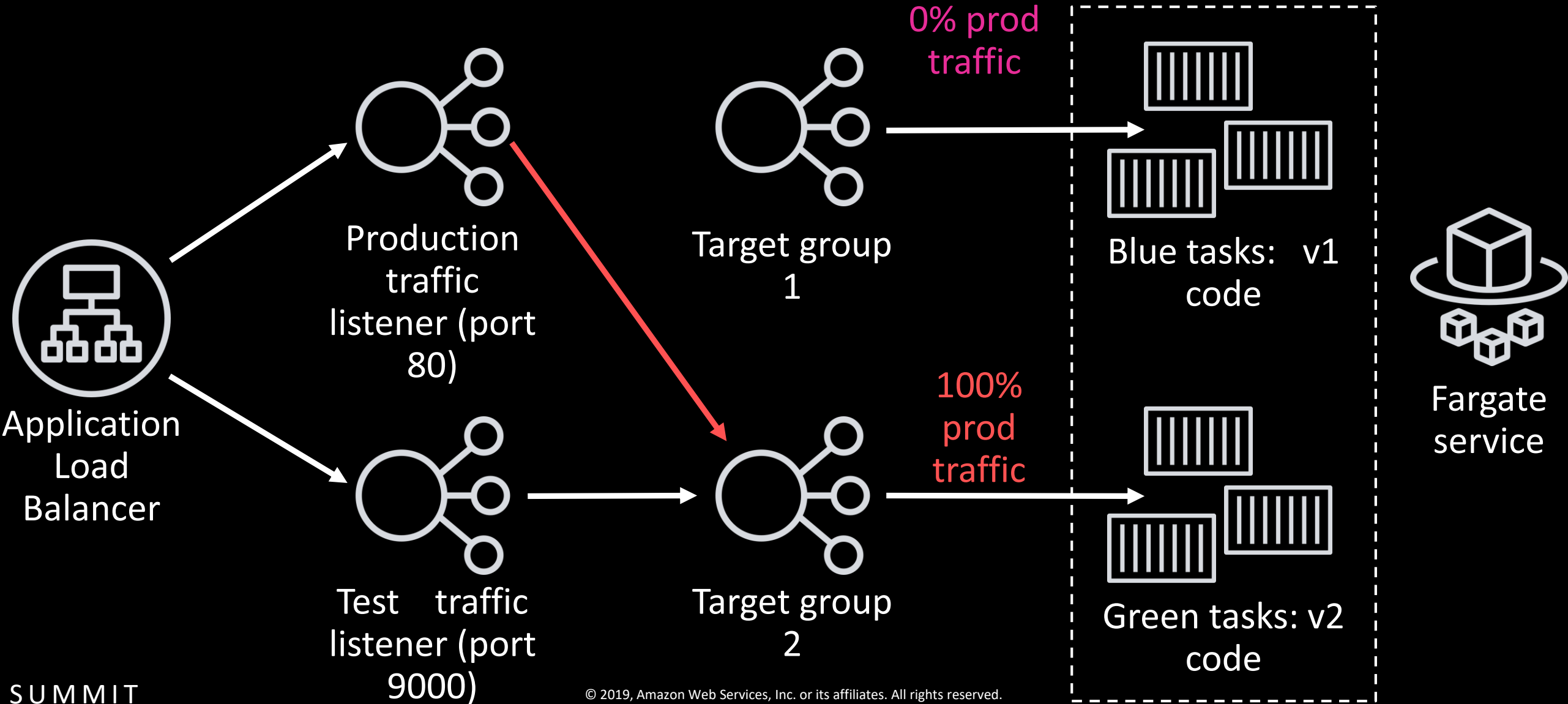
CodeDeploy-Amazon ECS blue/green deployment

Run hook against test endpoint before green tasks receive prod traffic



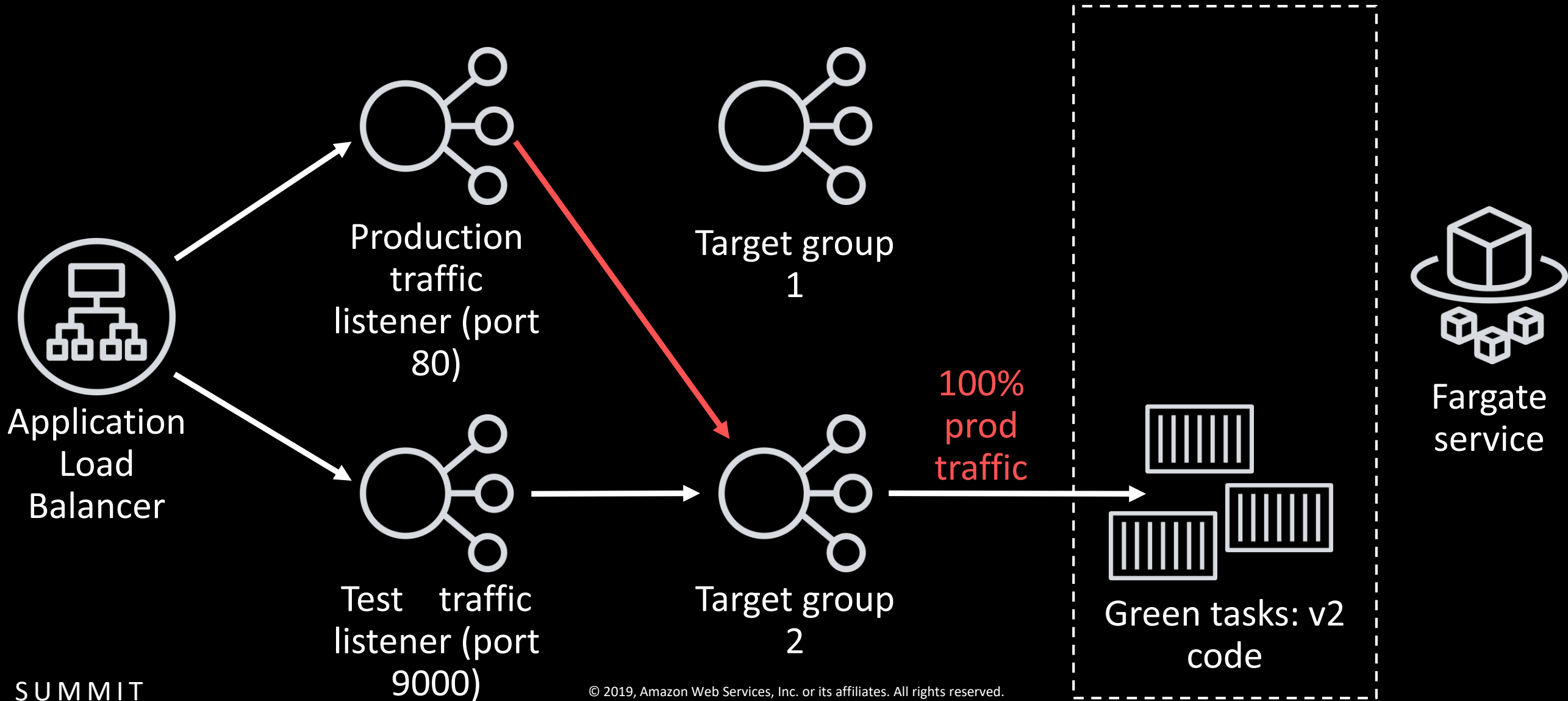
CodeDeploy-Amazon ECS blue/green deployment

Flip traffic to green tasks, roll back in case of alarm



CodeDeploy-Amazon ECS blue/green deployment

Drain blue tasks



Demo

Continuous-deployment goals



Continuous deployment

1. Automatically deploy new changes to staging environments for testing
2. Deploy to production safely without impacting customers
3. Deliver to customers faster: Increase deployment frequency, and reduce change lead time and change failure rate

Pillars of releasing modern applications

Continuous
integration

Continuous
deployment

The Salesforce CI/CD story

Benjamin Susman
Lead Member of Technical Staff
Salesforce Commerce Cloud

Who we are and what we do

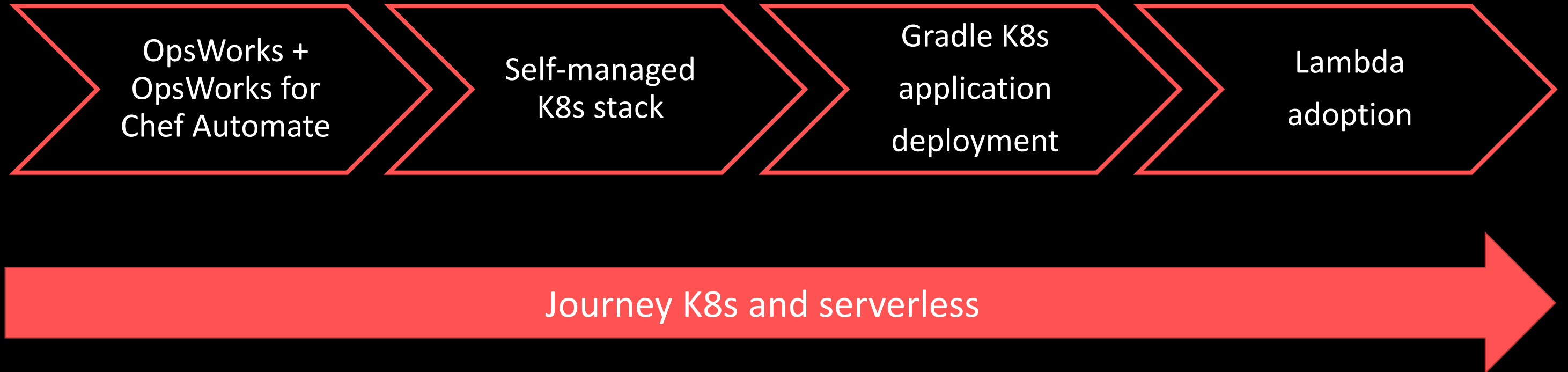
- We support **highly available monitoring solutions**
 - 30M+ metrics/minute consumed in custom time-series metrics store
 - Monitoring and real-time alerting service
 - Lambdas covering many areas, including:
 - Import and export of monitoring data to various central Salesforce subsystems
 - Complimentary service monitors
 - AWS admin and security automation
 - Event annotations support

Metrics by the numbers

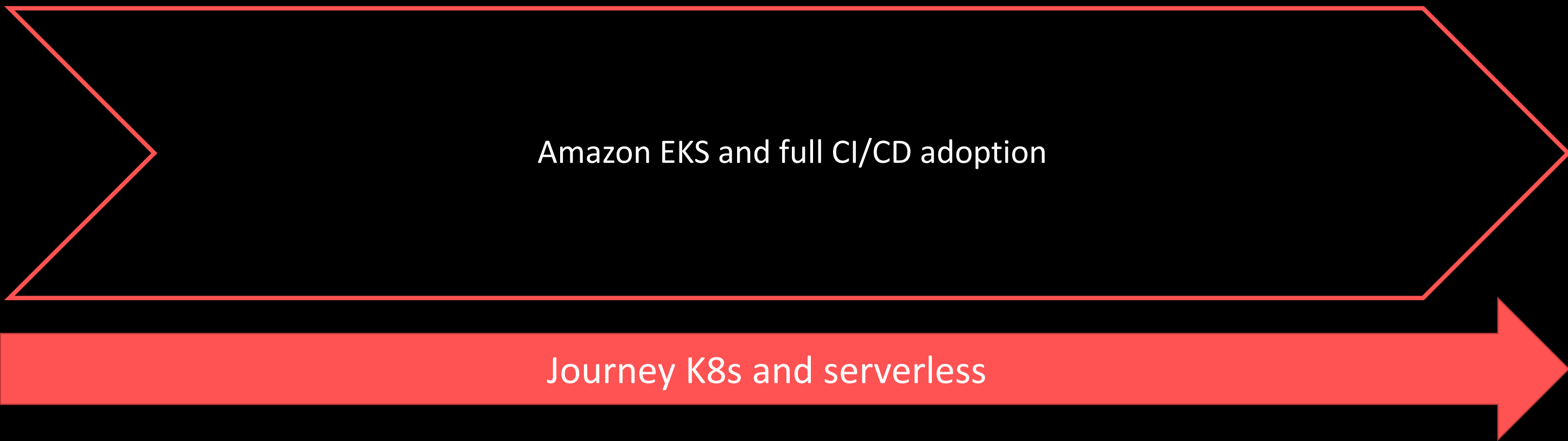


Source: 2018 DORA State of DevOps report

CI/CD journey



CI/CD journey



Amazon EKS and full CI/CD adoption

Journey K8s and serverless

Our use case

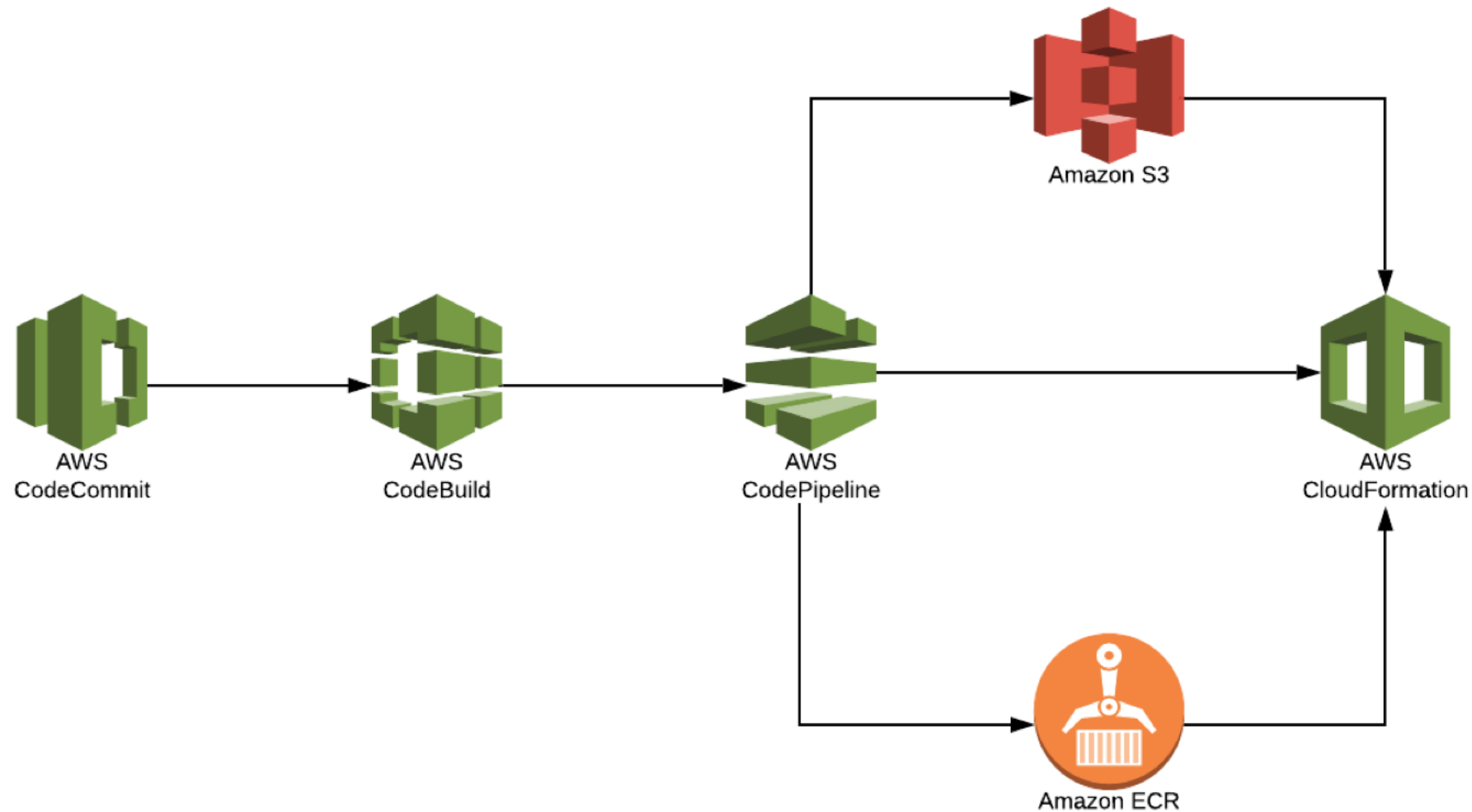
- Custom containerized apps in Amazon EKS
- Lambdas and Amazon ECS/Fargate
- Requirements and goals
 - AWS CloudFormation templates
 - Fully featured developer environment
 - Developer stacks separated from staging and production
 - All infrastructure in code
 - PR merge triggers automatic deployment of changes

Continuous-deployment pipeline I



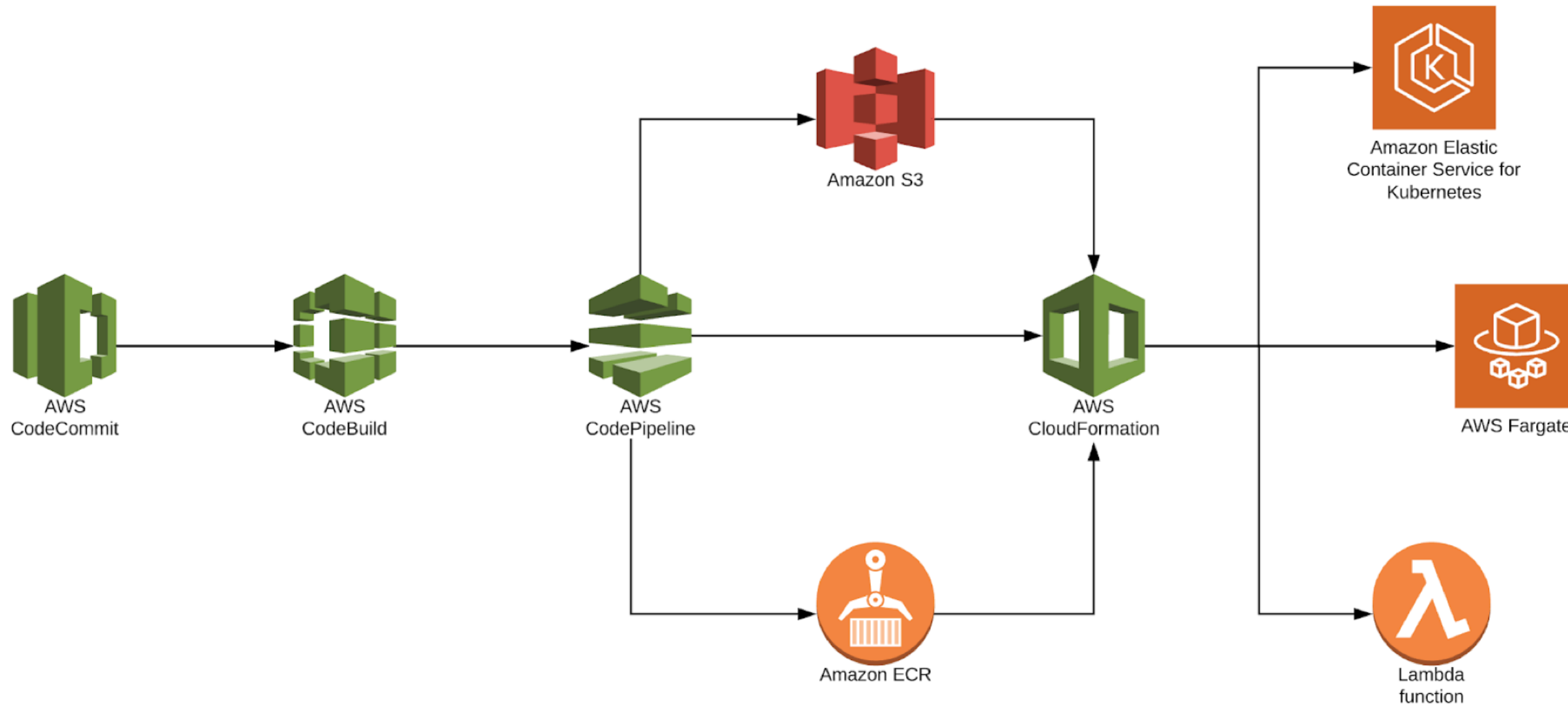
Source: 2018 DORA State of DevOps report

Continuous-deployment pipeline II



Source: 2018 DORA State of DevOps report

Continuous-deployment pipeline II



Source: 2018 DORA State of DevOps report

© 2019, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Continuous-deployment pipeline IV: Technologies

AWS CloudFormation

Infrastructure as code

Either all changes are applied or none

Easy to roll back

Clean deletes

CodePipeline

Just a resource in
AWS CloudFormation

Continuous-deployment pipeline V

Single repository for all applications

Infrastructure, configuration, app code in one place

Consistent developer experience

Code changes for apps, infra, config

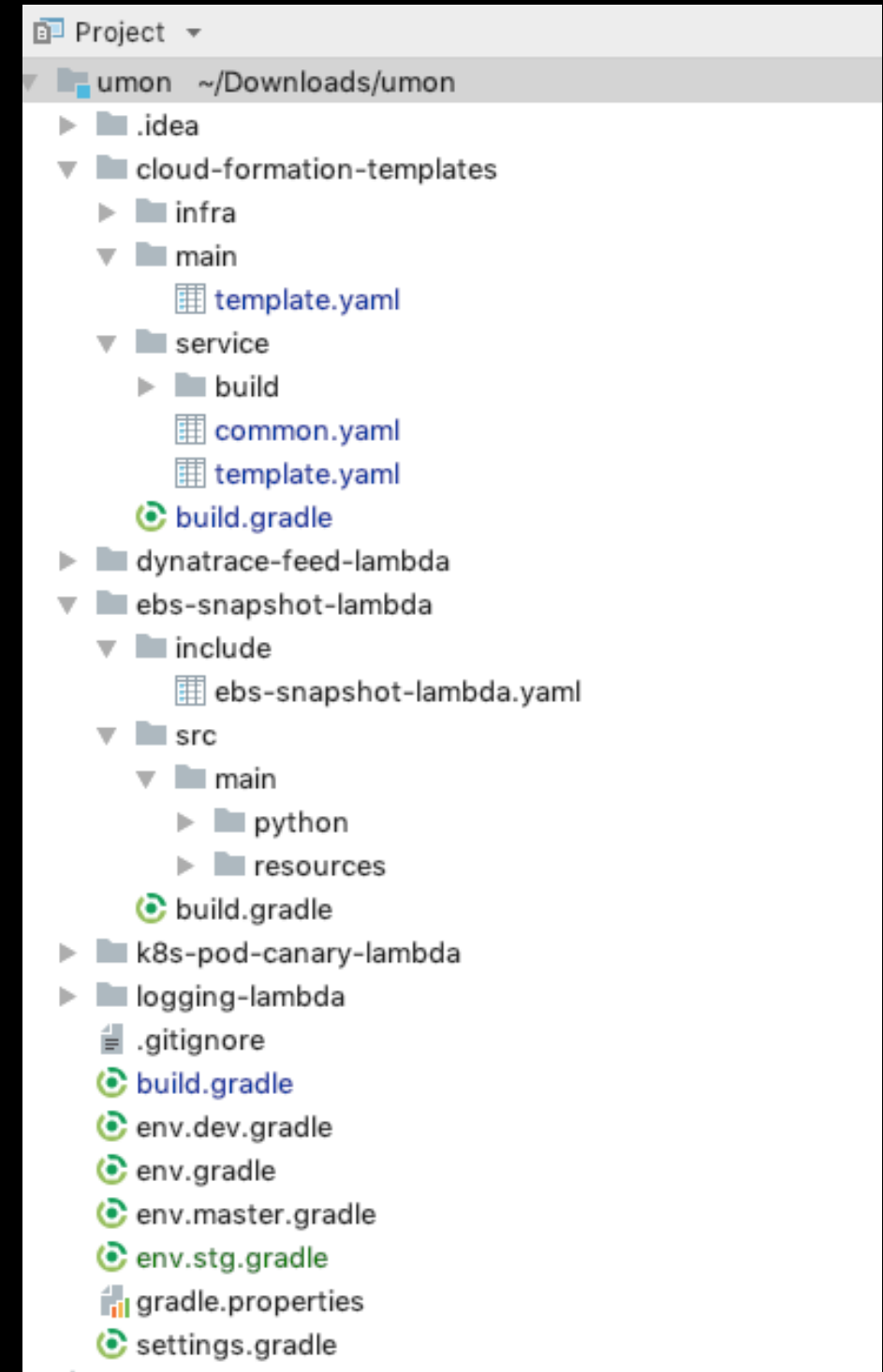
Different types of apps (lambda, container)

Different environments (stg/prd)

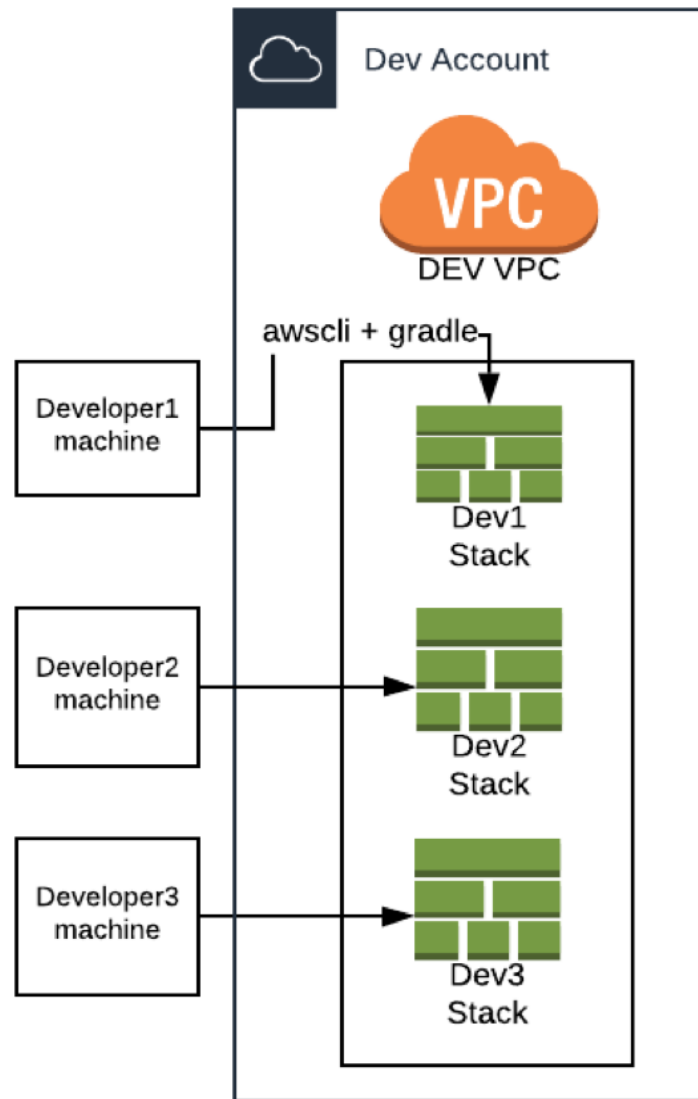
Every change is a code change

No manual changes, everything recorded

Infra and config is deployed the same way as code

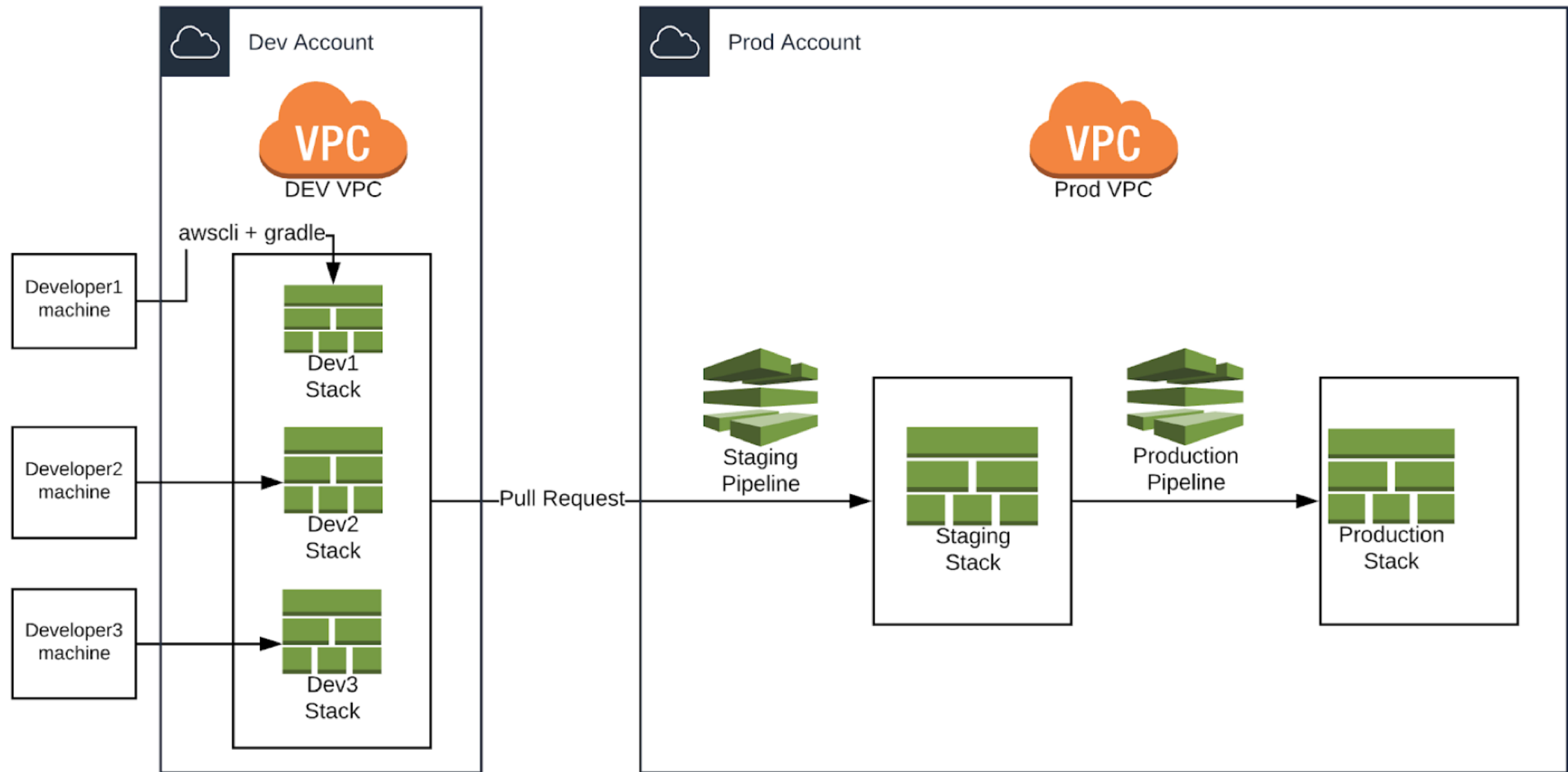


Amazon VPC and account structure I



Source: 2018 DORA State of DevOps report

Amazon VPC and account structure II

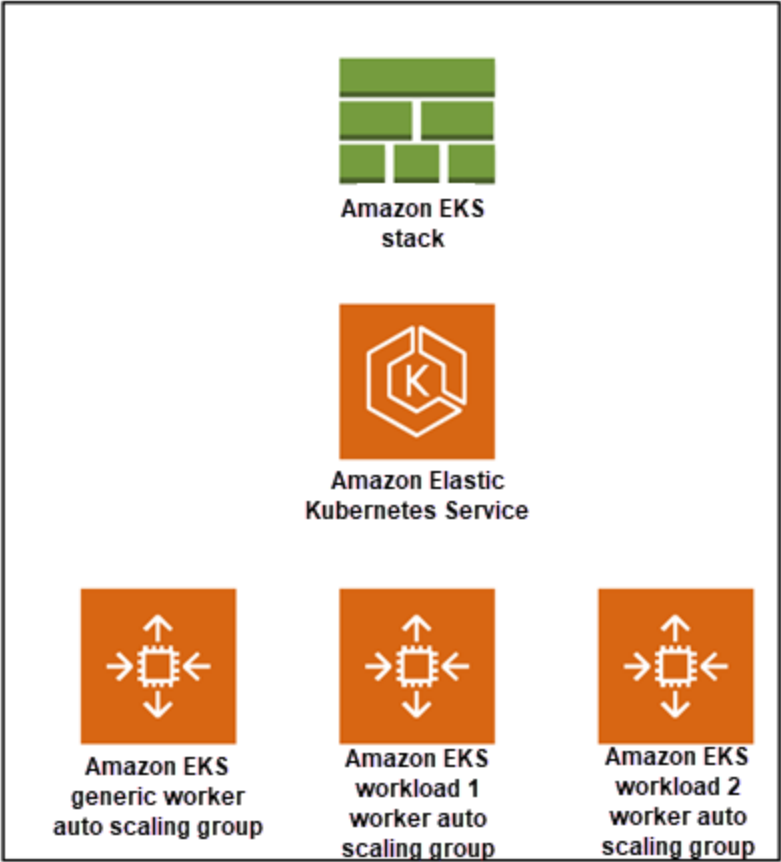
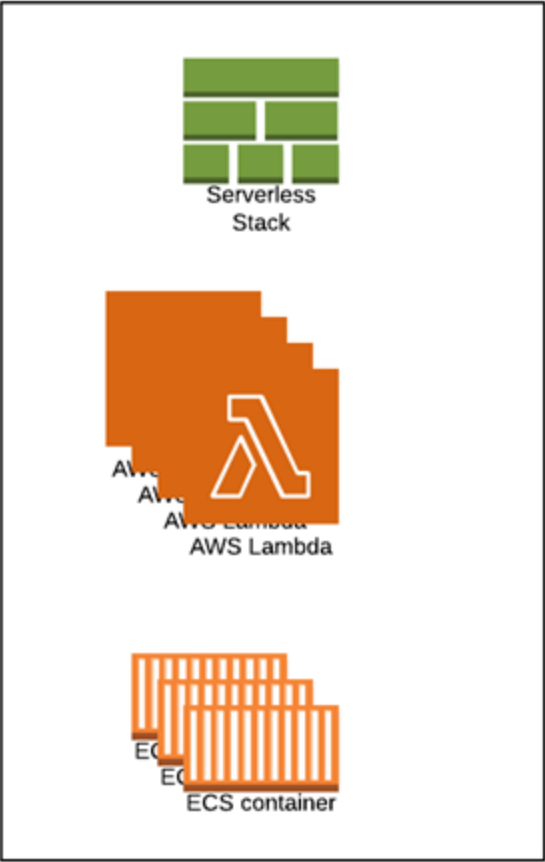
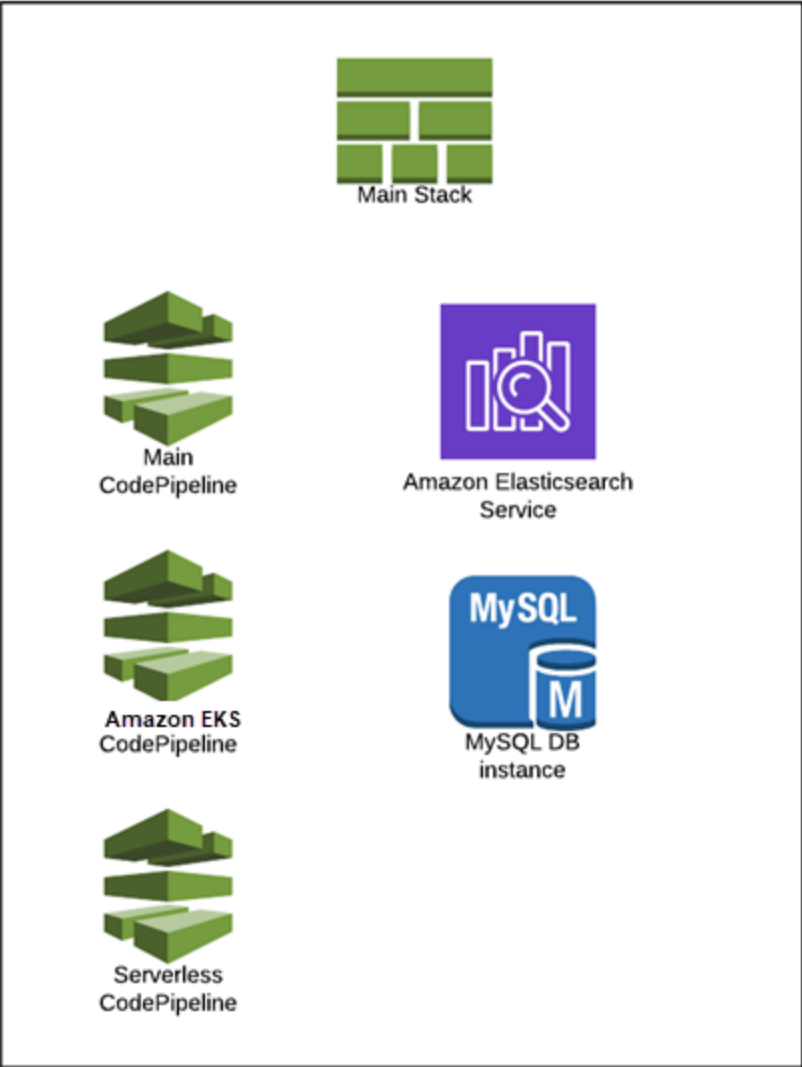


Source: 2018 DORA State of DevOps report

Amazon VPC and account structure III

- 2 accounts: DEV/PRD
- 2 VPCs: DEV/PRD
- PRD VPC also hosts staging environment
- Each developer gets their own dedicated stack
- Expensive resources shared between dev stacks (e.g., Amazon EKS)

Logical organization I

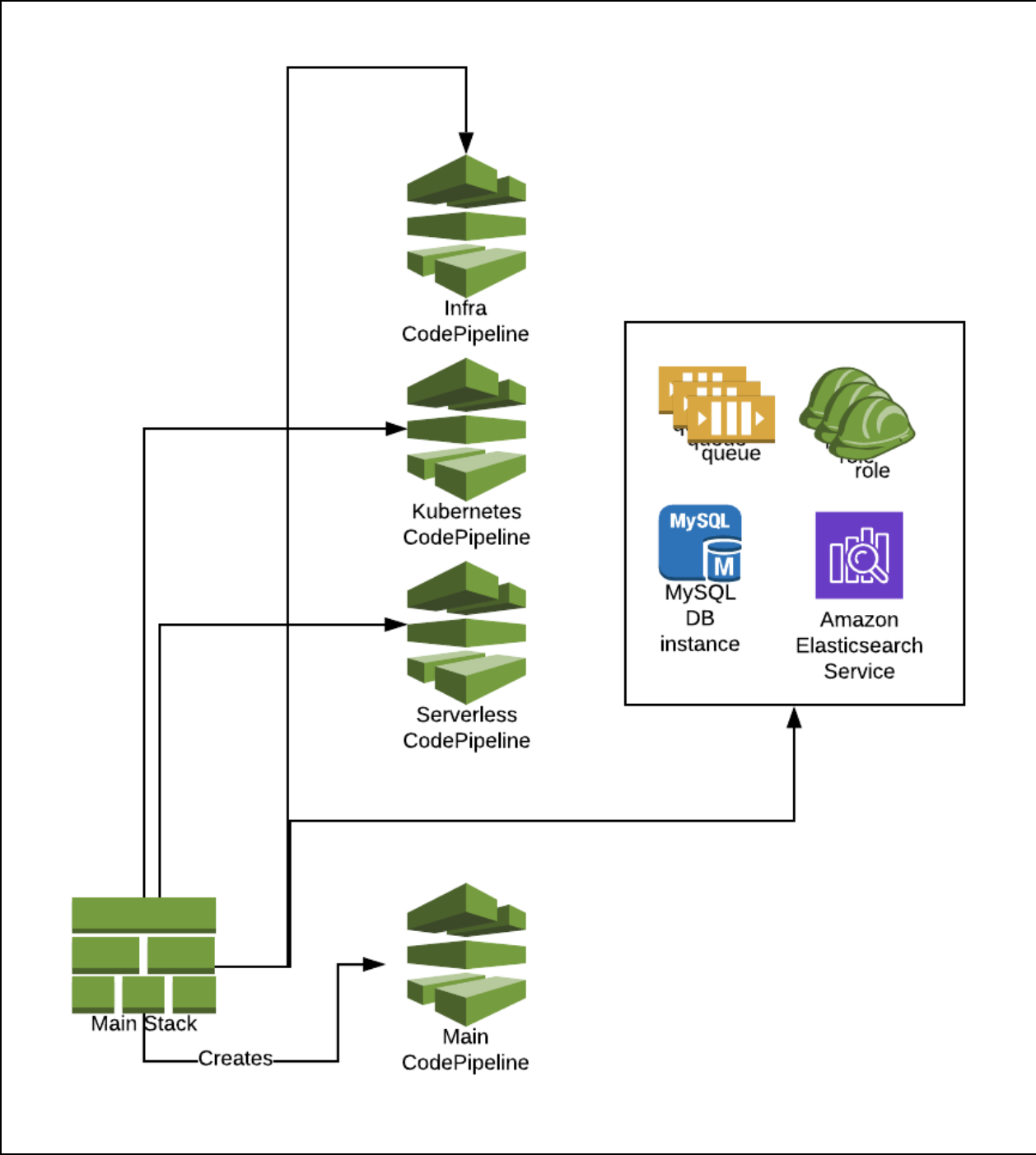


Source: 2018 DORA State of DevOps report

Logical organization II

- CloudFormation stacks
 - Stack is collection of AWS resources
 - All environments consist of three stacks
 - Main stack – some stateful resources, CodePipelines
 - Serverless stack – stateless resources
 - Amazon EKS stack – resources supporting and including Amazon EKS
- Why multiple stacks?
 - Group similar resources together
 - Main stack in order to be able to bootstrap

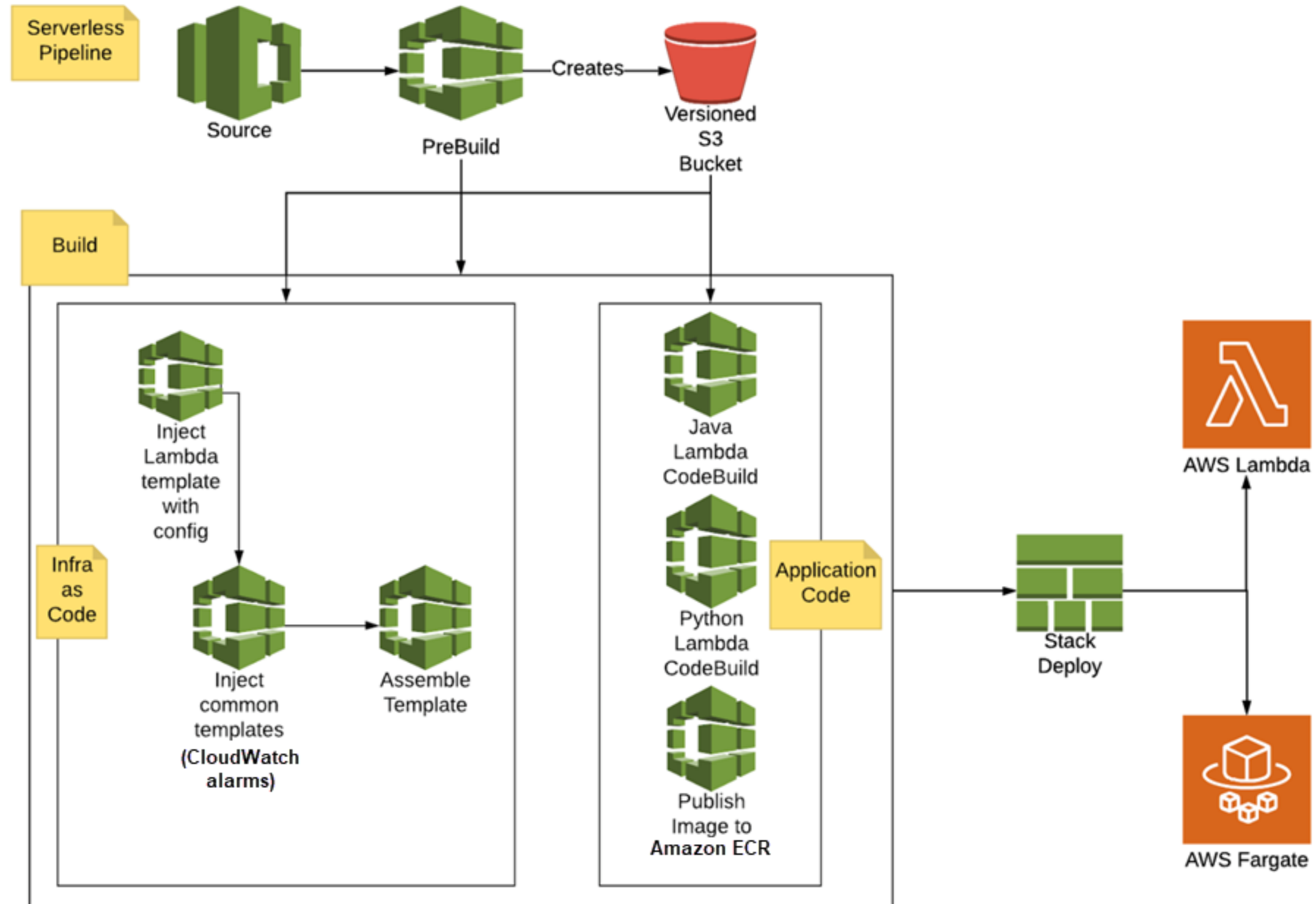
Main stack I – Bootstrap



Main stack II: Summary

- Used for **initial bootstrapping** of environments (DEV/STG/PRD)
- Creates CodePipelines for serverless, Amazon EKS applications
- Stack **changes are applied to itself** through AWS CloudFormation deployments
 - Automatically trigger rebuild

Serverless pipeline I



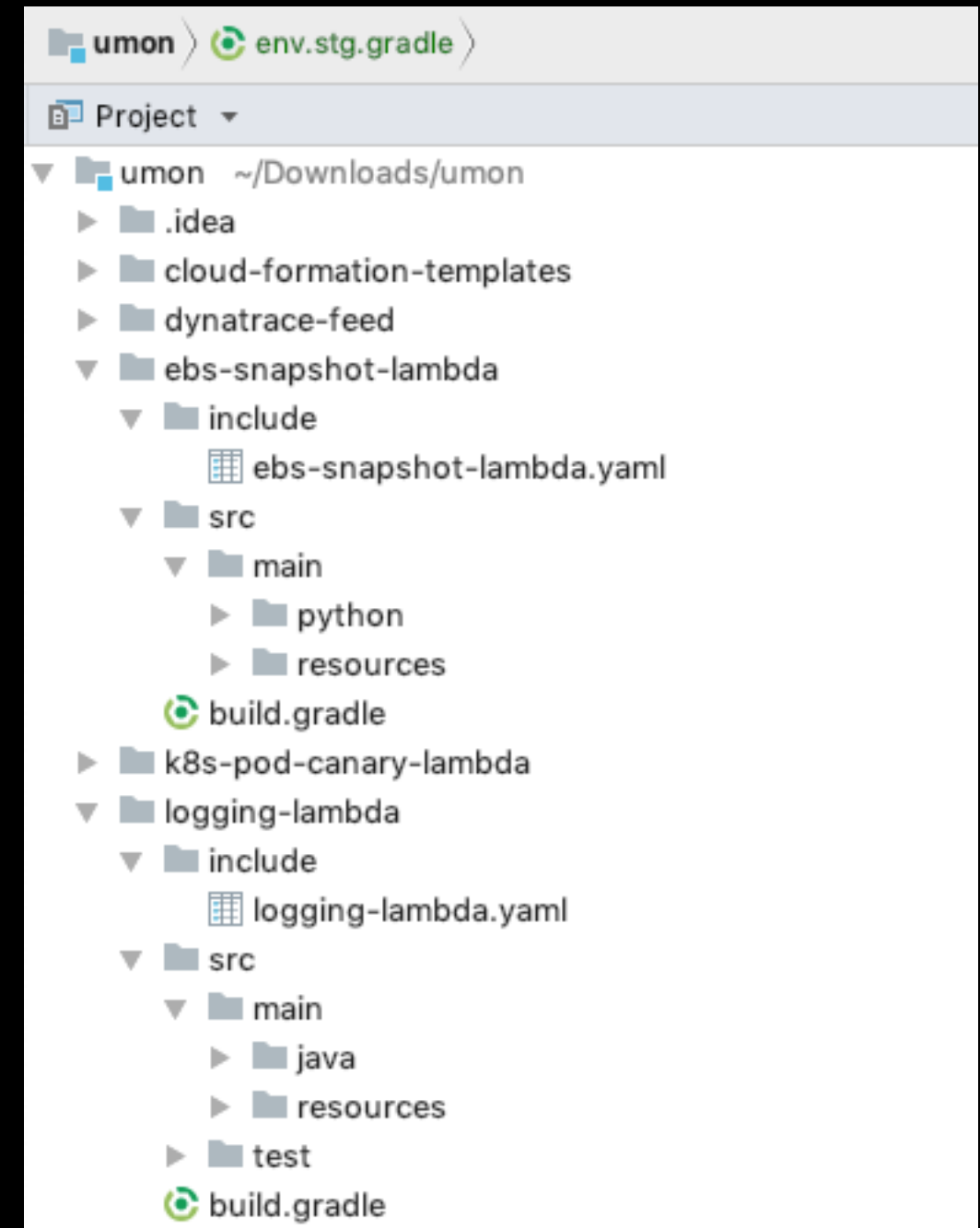
Serverless pipeline II

Lambda code bundled with infra code in one project

Each Lambda augmented with CloudWatch metrics, alerts, and notifications if Lambda fails

Avoids duplication of boilerplate code

Streamlines build environment between JS/Java/Python



Serverless pipeline III

Lambda is built and build artifact referenced in template for each Lambda

Each commit is a new version – build artifact changes based on commit hash

Redeployment of all Lambdas for each build

Cheap/fast

Deploy often, fail fast

Detects breaking changes

```
LoggingLambda:
  Type: AWS::Serverless::Function
  Properties:
    CodeUri:
      Bucket: !Ref s3bucket
      Key: !Sub "${s3Bucket}/${commitHash}/logging-lambda.zip"
    Events:
      Stream:
        Type: Kinesis
        Properties:
          Stream:
            "Fn::Sub":
              - arn:aws:kinesis:${AWS::Region}:${AWS::AccountId}:stream/${loggingStream}
              - loggingStream:
                  "Fn::ImportValue": !Sub ${Env}-LoggingStream
          StartingPosition: LATEST
          BatchSize: 25
      Handler: com.salesforce.cc.infra.logging.LoggingLambda
      MemorySize: 2048
      Runtime: java8
      Timeout: 120
```

Best practices I

Consistent developer experience

All infra, apps, and config in code make it easy to replicate full environment

Share costly resources

Deploy often, fail fast

One code repository for everything¹

AWS CloudFormation makes rollback atomic, cheap, and easy

Think about time savings as well—**45 minutes to production!**

<https://cacm.acm.org/magazines/2016/7/204032-why-google-stores-billions-of-lines-of-code-in-a-single-repository/fulltext>

Best practices II

Use tracing, CloudWatch Logs, and built-in metrics to monitor lambdas

More detailed metrics from lambdas: Leverage open-source monitoring tools when cost-effective

Create common code packages to share across lambdas (all code in one repo)

Provide logical defaults with the ability to easily override

Externalize configuration for easier templating environments (DEV/STG/PRD)

Set concurrency limit for lambdas wherever applicable

Best practices III

Developers are lazy

- Limit number of costly resources to absolute minimum for DEV/STG (EC2 instances, etc.)
- Delete unused resources and automatically

Common pitfalls/lessons learned

- AWS CloudFormation updates should be tested in the CI/CD pipeline
- AWS CloudFormation feature support
- Not all resources can be updated easily (e.g., immutable NLBs, DNS names)
- Stack names have to be unique – issue for dev environments

Thank you!